

目录

后缀自动机	1
广义后缀自动机	2
Ac 自动机	4
回文自动机	6
最小表示法	7
Manacher	7
Z 函数	8
Kmp	8
Kmp 自动机	9
后缀数组	9
Lyndon 分解	10
通配符	11

后缀自动机

```
long long ans=0;
struct SAutomatons{
    struct Nodes{
        int link,len,siz;
        std::array<int,26>Son;
        Nodes():link(0),len(0),siz(0){
            std::fill(Son.begin(),Son.end(),0);
        }
    };
    vector<Nodes>N;
    vector<vector<int>>>To;
    int tot,last;
    SAutomatons(const int&l){Clear(l);}
    void Clear(const int&l){
        Clr(N);
        N.resize(2*l+10);//预防边界情况
        To.resize(2*l+10);//预防边界情况
        tot=last=0;
        N[last].link=-1;
    }
    void Add(int c){
        int now=last;N[last=++tot].siz=1;N[last].len=N[now].len+1;
        for(;now!=-1&&N[now].Son[c]==0;now=N[now].link)
            N[now].Son[c]=tot;
        if(now==-1)N[last].link=0;
        else{
            int tmp=N[now].Son[c];
```

```

        if(N[tmp].len==N[now].len+1)N[last].link=tmp;
        else{
            int newtmp=++tot;
            N[newtmp].len=N[now].len+1;
            std::copy(N[tmp].Son.begin(), N[tmp].Son.end(), N[newtmp].Son.begin());
            N[newtmp].link=N[tmp].link;
            N[tmp].link=newtmp;
            N[last].link=newtmp;
            for(;now!=-1&&N[now].Son[c]==tmp;now=N[now].link)N[now].Son[c]=newtmp;
        }
    }
}
void Build(){
    rep(i,1,tot)To[N[i].link].push_back(i);
}
void Dfs(int from){
    for(auto t:To[from]){
        Dfs(t);
        N[from].siz+=N[t].siz;
    }
    if(N[from].siz!=1){
        ans=std::max(ans,1ll*N[from].siz*N[from].len);
    }
}
};
SAutomatons SAM(1000000);
int n;char St[1100000];
signed main(){
    scanf("%s",St+1);
    n=strlen(St+1);
    for(int i=1;i<=n;i++)SAM.Add(St[i]-'a');
    SAM.Build();
    SAM.Dfs(0);
    printf("%lld",ans);
    return 0;
}

```

广义后缀自动机

```

struct GsAutomatons{
    struct Tries{
        struct Nodes{
            int dad;
            char which;
            std::array<int,26>Son;
            Nodes(){
                std::fill(Son.begin(),Son.end(),0);
            }
        };
        vector<Nodes>T;
        int cnt;
        Tries(const int&l){Clear(l);}
    };
};

```

```

void Clear(const int&l){
    cnt=0;
    //Clr(T);
    T.resize(l+10);
}
}Tr;
struct Nodes{
    int link,len,pos;
    std::array<int,26>Son;
    Nodes():link(0),len(0),pos(0){
        std::fill(Son.begin(),Son.end(),0);
    }
};
vector<Nodes>N;
int tot;
GsAutomatons(const int&l):Tr(l){Clear(l);}
void Clear(const int&l){
    //Tr.Clear(L);
    tot=0;
    Clr(N);
    N.resize(2*l+10);
    N[0].link=-1;
}
void Insert(const std::string&S){
    int now=0;
    for(auto s:S){
        if(!Tr.T[now].Son[s-'a']){
            Tr.T[now].Son[s-'a']=++Tr.cnt;
            Tr.T[Tr.cnt].dad=now;
            Tr.T[Tr.cnt].which=s-'a';
        }
        now=Tr.T[now].Son[s-'a'];
    }
}
int Add(int c,int last){
    int now=last;last=++tot;
    N[last].len=N[now].len+1;
    for(;now!=-1&&N[now].Son[c]==0;now=N[now].link)
        N[now].Son[c]=tot;
    if(now==-1)N[last].link=0;
    else{
        int tmp=N[now].Son[c];
        if(N[tmp].len==N[now].len+1)N[last].link=tmp;
        else{
            int newtmp=++tot;
            std::copy(N[tmp].Son.begin(), N[tmp].Son.end(), N[newtmp].Son.begin());
            N[newtmp].len=N[now].len+1;
            N[newtmp].link=N[tmp].link;
            N[tmp].link=newtmp;
            N[last].link=newtmp;
            for(;now!=-1&&N[now].Son[c]==tmp;now=N[now].link)N[now].Son[c]=newtmp;
        }
    }
}

```

```

        return last;
    }

    void Build(){
        std::queue<int>,std::list<int>>Q;
        for(int i=0;i<=25;i++){
            if(Tr.T[0].Son[i])Q.push(Tr.T[0].Son[i]);
            while(!Q.empty()){
                int from=Q.front();Q.pop();
                N[from].pos=Add(Tr.T[from].which,N[Tr.T[from].dad].pos);
                for(int i=0;i<=25;i++){
                    if(Tr.T[from].Son[i])Q.push(Tr.T[from].Son[i]);
                }
            }
        }
    };GsAutomatons GSAM(1000010);
    void Main(int Case){
        int n;InPut(n);
        std::string S;
        for(int i=1;i<=n;i++){
            InPut(S);
            GSAM.Insert(S);
        }
        GSAM.Build();
        long long ans=0;
        for(int i=1;i<=GSAM.tot;i++){
            ans+=GSAM.N[i].len-GSAM.N[GSAM.N[i].link].len;
        }
        OutPut(ans,'\n');
        OutPut(GSAM.tot+1,'\n');
    }
}

```

Ac 自动机

```

vector<long long>Ans;
vector<int>Pos;
struct AcAutomatons{
    struct Nodes{
        int fail,siz;
        std::array<int,26>Son;
        Nodes():fail(0),siz(0){
            std::fill(Son.begin(),Son.end(),0);
        }
    };
    vector<Nodes>N;
    vector<vector<int>>>To;
    int tot;
    AcAutomatons(const int&l){Clear(l);}
    void Clear(const int&l){
        //Clr(N);
        N.resize(l+10);
        To.resize(l+10);
        N[0].fail=-1;
    }
}

```

```

        tot=0;
    }
    void Insert(const std::string&S,int id){
        int now=0;
        for(auto s:S){
            if(N[now].Son[s-'a']==0)
                N[now].Son[s-'a']=++tot;
            now=N[now].Son[s-'a'];
        }
        Pos[id]=now;
    }
    void Build(){
        std::queue<int>,std::list<int>>Q;
        for(int i=0;i<=25;i++)
            if(N[0].Son[i])N[N[0].Son[i]].fail=0,Q.push(N[0].Son[i]);
        while(!Q.empty()){
            int fr=Q.front();Q.pop();
            for(int i=0;i<=25;i++){
                if(N[fr].Son[i]){
                    N[N[fr].Son[i]].fail=N[N[fr].fail].Son[i];
                    Q.push(N[fr].Son[i]);
                }
                else N[fr].Son[i]=N[N[fr].fail].Son[i];
            }
        }
    }
    void BuildT(){
        rep(i,1,tot)
            To[N[i].fail].push_back(i);
    }
    void Query(const std::string&S){
        int now=0;
        for(auto s:S){
            now=N[now].Son[s-'a'];
            Ans[now]++;
        }
    }
    void Dfs(int from){
        for(auto to:To[from]){
            Dfs(to);
            Ans[from]+=Ans[to];
        }
    }
};AcAutomatons acam(210000);

void Main(int Case){
    Ans.resize(210000);
    int n;InPut(n);
    Pos.resize(n+1);

    for(int i=1;i<=n;i++){
        std::string S;
        InPut(S);
    }
}

```

```

        acam.Insert(S,i);
    }

    std::string T;
    InPut(T);
    acam.Build();
    acam.BuildT();
    acam.Query(T);
    acam.Dfs(0);

    for(int i=1;i<=n;i++)
        OutPut(Ans[Pos[i]],'\n');
}

```

回文自动机

```

#include<cstdio>
#include<cstring>
#include<algorithm>
using namespace std;

template<int MaxN>
struct PAMs{
    char St[MaxN];int len;
    int Len[MaxN]; //回文串长度
    int Son[MaxN][26]; //转移边，加一个字符还是回文
    int Fail[MaxN]; //最长回文后缀
    int Sum[MaxN];
    int Link[MaxN][26]; //Link[x][c] 表示节点 x 所代表的的串向左扩展一个字符 c 找到的串。
    int last,cnt;
    PAMs(){
        cnt=1;
        Fail[0]=1; //偶回文中心
        for(int i=0;i<=25;i++)
            Link[1][i]=1;
        Fail[1]=1; //奇回文中心
        for(int i=0;i<=25;i++)
            Link[0][i]=1;
        Len[1]=-1;
    }
    inline void Add(char c){
        St[++len]=c;
        int now=last,x=St[len]-'a';
        if(St[len-Len[now]-1]!=St[len])now=Link[now][x];
        if(!Son[now][x]){
            int np=++cnt;
            Len[np]=Len[now]+2;
            Fail[np]=Son[Link[now][x]][x];
            memcpy(Link[np],Link[Fail[np]],sizeof(Link[Fail[np]]));
            Link[np][St[len-Len[Fail[np]]]-'a']=Fail[np];
            Sum[np]=Sum[Fail[np]]+1;
        }
        last=np;
    }
};

```

```

        Son[now][x]=np;
    }
    last=Son[now][x];
}
};PAMs<510000>PAM;
int last;char c;
int main(){
    while((c=getchar())!='\r'){
        c=(c-97+last)%26+97;
        PAM.Add(c);
        printf("%d ",last=PAM.Sum[PAM.last]);
    }
    return 0;
}

```

最小表示法

```

int k = 0, i = 0, j = 1;
while (k < n && i < n && j < n) {
    if (sec[(i + k) % n] == sec[(j + k) % n]) {
        k++;
    } else {
        sec[(i + k) % n] > sec[(j + k) % n] ? i = i + k + 1 : j = j + k + 1;
        if (i == j) i++;
        k = 0;
    }
}
i = min(i, j);

```

Manacher

```

int main(){
    scanf("%s",St+1);n=strlen(St+1);
    Tmp[++len]='?';
    for(int i=1;i<=n;i++){
        Tmp[++len]='#';
        Tmp[++len]=St[i];
    }
    Tmp[++len]='#';
    Tmp[++len]='&';
    int mid=1,ans=0;
    Res[1]=0;
    for(int i=2;i<=len-2;i++){
        int right=mid+Res[mid];
        if(i<=right)
            Res[i]=min(i+Res[2*mid-i],right)-i;
        while(Tmp[i-Res[i]-1]==Tmp[i+Res[i]+1])
            ++Res[i];
    }
}

```

```

        if(i+Res[i]>=mid)mid=i;
        ans=max(ans,Res[i]);
    }
    printf("%d\n",ans);
    return 0;
}

```

Z 函数

```

int main(){
    scanf("%s",A+1);
    la=strlen(A+1);
    scanf("%s",B+1);
    lb=strlen(B+1);
    for(int i=1;i<=lb;i++)
        C[i]=B[i];
    for(int i=1;i<=la;i++)
        C[i+lb]=A[i];
    //拼接是常用的技巧 当然用 z of B 求出 z of A 也可行
    Zi[1]=la+lb-1;
    for(int i=2,l=0,r=0;i<=la+lb;i++){
        if(i<=r&&Zi[i-l+1]<=r-i)
            Zi[i]=Zi[i-l+1];
        else{
            Zi[i]=max(0,r-i);
            while(i+Zi[i]<=la+lb&&C[Zi[i]+1]==C[i+Zi[i]])
                Zi[i]++;
        }
        if(i+Zi[i]-1>r)l=i,r=i+Zi[i]-1;
    }
    ans=0;
    for(int i=1;i<=lb;i++)
        ans^=1ll*i*(min(Zi[i],lb-i+1)+1);
    printf("%lld\n",ans);
    ans=0;
    for(int i=lb+1;i<=lb+la;i++)
        ans^=1ll*(i-lb)*(min(Zi[i],lb)+1);
    printf("%lld\n",ans);
    return 0;
}

```

Kmp

```

int main(){
    scanf("%s",B+1);m=strlen(B+1);
    scanf("%s",A+1);n=strlen(A+1);
    int j;
    j=0;

```



```

for(int i=2;i<=n;i++){
    while(j>0&&A[i]!=A[j+1])j=Next[j];
    if(A[i]==A[j+1])j++;
    Next[i]=j;
}
j=0;
for(int i=1;i<=m;i++){
    while(j>0&&(B[i]!=A[j+1]||j==n))j=Next[j];
    if(B[i]==A[j+1])j++;
    if(j==n)printf("%d\n",i-n+1);
}
j=0;

for(int i=1;i<=n;i++)
    printf("%d ",Next[i]);

return 0;
}

```

Kmp 自动机

```

for(int i = 1, fail = 0; i <= n; i ++ ) {
    fail = nxt[fail][s[i]]; // 注意这一行不能和下一行互换
    nxt[i - 1][s[i]] = i;
    for(int j = 0; j < m; j ++ )
        nxt[i][j] = nxt[fail][j];
}

```

后缀数组

```

struct SaArray{
    SaArray()=delete;
    SaArray(const char*st):S(st){}
    SaArray(const std::string&st):S(st){}
    std::string S;
    int n,m;
    std::vector<int>Sa,Id,Rk,Lc;
    void getSa(){
        n=S.size()-1;m=128;
        Sa.resize(n+10);
        Id.resize(n+10);
        Rk.resize(n+10);
        Lc.resize(n+10);
        int p;
        std::vector<int>Cnt(2100000);
        std::vector<int>OlRk(2*n+10);
        for(int i=1;i<=n;i++)Cnt[Rk[i]=S[i]]++;
        for(int i=1;i<=m;i++)Cnt[i]+=Cnt[i-1];
        for(int i=n;i>=1;i--)Sa[Cnt[Rk[i]]--]=i;
        for(int i=1;i<=m;i++)Cnt[i]=0;
    }
}

```

```

for(int w=1;w<=n;w<=1,m=p){
    int cur=0;
    for(int i=n-w+1;i<=n;i++)Id[++cur]=i;//这些第二关键字都是 0
    for(int i=1;i<=n;i++)
        if(Sa[i]>w)Id[++cur]=Sa[i]-w;//第二关键字排完序将自身的第一关键字扔进队列
    for(int i=1;i<=n;i++)Cnt[Rk[Id[i]]]++;
    for(int i=1;i<=m;i++)Cnt[i]+=Cnt[i-1];
    for(int i=n;i>=1;i--)Sa[Cnt[Rk[Id[i]]]--]=Id[i];
    for(int i=1;i<=m;i++)Cnt[i]=0;
    p=0;
    for(int i=1;i<=n;i++)
        OlRk[i]=Rk[i];
    for(int i=1;i<=n;i++){
        if(OlRk[Sa[i]]==OlRk[Sa[i-1]]&&OlRk[Sa[i]+w]==OlRk[Sa[i-1]+w])
            Rk[Sa[i]]=p;
        else Rk[Sa[i]]==++p;
    }
    if(p==n)break;
}
for(int i=1,k=0;i<=n;i++){//Lc[Rk[i]]>=Lc[Rk[i-1]]-1
    if(Rk[i]==0)continue;
    if(k)--k;
    while(S[i+k]==S[Sa[Rk[i]-1]+k])++k;
    Lc[Rk[i]]=k;
}
//Lcp(Sa[i],Sa[j])=min_{i+1~j}Lc[k]
}
};

```

Lyndon 分解

```

vector<string> duval(string const& s) {
    int n = s.size(), i = 0;
    vector<string> factorization;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k) {
            factorization.push_back(s.substr(i, j - k));
            i += j - k;
        }
    }
    return factorization;
}

```

通配符

定义字符串下标从 0 开始，给定文本串 A 长度为 n ，模式串 B 长度为 m ，我们可以考虑一个函数 $F(x, y) = A(x) - B(y)$ 。

我们设 $F(x, y \geq m-1) = \sum_{i=0}^{m-1} f(x-m+1+i, i)$ ，由定义显然可以得到如果 $F(x) = 0$ ，则 $A[x-m+1, x]$ 也就是两个字符串匹配上了，但是考虑 “ab”，“ba” 两个字符串，他们是匹配的，我们稍微修改一下 $f(x, y)$ 函数令其为： $(A(x) - B(y))^2$ ，这样这个函数就没有问题了。

我们考虑翻转一下 B 串，令其为 S ，则有 $B(i) = S(m-1-i)$ 。

$$F(x) = \sum_{i=0}^{m-1} (A(x-m+1+i) - S(m-1-i))^2$$
$$F(x) = \sum_{i=0}^{m-1} S(m-1-i)^2 + \sum_{i=0}^{m-1} A(x-m+1+i)^2 - 2 \times \sum_{i=0}^{m-1} A(x-m+1+i)S(m-1-i)$$

第一项是一个定值，第二项可以 $O(n)$ 预处理，然后前缀和 $O(1)$ 得到，第三项不难发现是一个卷积的形式，所以可以通过 FFT 得到，整体复杂度 $O(n \log n)$ 。

以上我们已经可以解决模式串的字符串匹配了，尽管复杂度不如 KMP 优秀，但是我们考虑一个缺项字符串匹配：

a*b
aeb*ob

我们考虑重新设计 $f(x, y)$ 函数，定义 $f(x, y) = (A(x) - B(y))^2 A(x)B(y)$ ，同样的考虑翻转 B 串，有 $B(i) = S(m-1-i)$ 。

$$F(x) = \sum_{i=0}^{m-1} (A(x-m+1+i) - S(m-1-i))^2 A(x-m+1+i)S(m-1-i)$$
$$= \sum_{i=0}^{m-1} A(x-m+1+i)S(m-1-i)^3 + \sum_{i=0}^{m-1} A(x-m+1+i)^3 S(m-1-i) - 2 \times \sum_{i=0}^{m-1} A(x-m+1+i)^2 S(m-1-i)$$

容易发现这里是三个多项式相加的形式，所以只需要做三次 FFT 即可得到答案，放一个模板吧。